XPath Functions Cheat Sheet

Playwright Edition — every XPath function that actually works in the browser

27 CORE FUNCTIONS · XPATH 1.0 (BROWSER-NATIVE) · ALSO VALID IN SELENIUM & APPIUM

Playwright hands every XPath expression to the browser's native engine (document.evaluate), which implements **XPath 1.0**. The functions below are therefore the **complete** set that works in Playwright — and the same set works in Selenium and Appium, since they rely on the same engine. XPath 2.0+ functions such as ends-with() or matches() simply fail in the browser (workarounds in section 6).

§ Using XPath in Playwright

```
page.locator('xpath=//button[@type="submit"]');   // explicit engine prefix
page.locator('//button[@type="submit"]');         // auto-detected: starts with //
row.locator('..');                                  // chained: hop to the parent element
```

- Selectors starting with `//` or `..` are auto-detected as XPath — no prefix needed.
- XPath does **not pierce shadow DOM** in Playwright. For shadow-root content, use CSS or getByRole() / getByText().
- Prototype expressions fast in the DevTools Console: `$x('//button[contains(., "Save")]')`
- Playwright's own guidance: prefer user-facing locators (getByRole, getByText, getByLabel); reach for XPath when structure-based selection is unavoidable — tables, SVG, legacy DOMs.

The chaining gotcha (top interview + code-review catch):

```
const table = page.locator('#orders');
table.locator('xpath=.//tr[last()]');  [OK] searches inside #orders
table.locator('xpath=//tr[last()]');   [!!] silently searches the WHOLE page
```

1 String functions (the workhorses)

Function	Returns	Example	Notes
contains(s1, s2)	*boolean*	`//button[contains(., 'Save')]`	The most-used XPath function. "." = the element's full text, including nested tags.
starts-with(s1, s2)	*boolean*	`//div[starts-with(@id, 'react-select-')]`	Your escape hatch for auto-generated / dynamic IDs.
normalize-space(s?)	*string*	`//button[normalize-space()='Sign in']`	Trims ends + collapses inner whitespace. The safest exact-text match.
concat(s1, s2, ...)	*string*	`//button[text()=concat("Don", "'", "t save")]`	Also the standard trick for target text that contains quotes.
substring(s, start, len?)	*string*	`substring('2026-07-12', 1, 4) -> '2026'`	Index is 1-based, not 0-based — a classic interview trap.
substring-before(s, d)	*string*	`substring-before('user@test.io', '@') -> 'user'`	Returns an empty string if the delimiter is not found.
substring-after(s, d)	*string*	`substring-after('order-4217', '-') -> '4217'`	Great for peeling values off dynamic IDs.
string-length(s?)	*number*	`//input[string-length(@value) > 0]`	No-argument form measures the context node's own text.
translate(s, from, to)	*string*	`see pattern #1 (case-insensitive match)`	Char-by-char mapping. Chars in 'from' with no partner in 'to' are deleted.
string(obj?)	*string*	`//li[string(.)='Done']`	string(.) = concatenated text of the element and all of its descendants.

2 Node & position functions

Function	Returns	Example	Notes
`position()`	*number*	`//tr[position() <= 3]`	1-based position in the current node list. //tr[2] is shorthand for //tr[position()=2].
`last()`	*number*	`//li[last()]` `//li[last()-1]`	The last (and second-to-last) matching element.
`count(node-set)`	*number*	`//ul[count(li) > 5]`	Filter containers by how many children they have.
`name(node?)`	*string*	`//*[name()='svg']`	THE SVG TRICK: SVG lives in its own namespace, so //svg finds nothing — this does.
`local-name(node?)`	*string*	`//*[local-name()='path' and @fill='red']`	Like name() but strips namespace prefixes. Safest for elements inside an SVG.
`namespace-uri(node?)`	*string*	`//*[namespace-uri()='http://www.w3.org/2000/svg']`	Rare in UI tests; more useful in XML / API payloads.
`id(str)`	*node-set*	`id('login-form')`	Legacy. Prefer //*[@id='login-form'] or CSS #login-form.

3 Boolean functions

Function	Returns	Example	Notes
`not(expr)`	*boolean*	`//input[not(@disabled)]`	Second-most-used function: enabled fields, //li[not(contains(@class,'done'))].
`boolean(obj)`	*boolean*	`//button[boolean(@data-loaded)]`	Truthiness: node-set is non-empty / string is non-empty / number is not 0 or NaN.
`true() / false()`	*boolean*	`//button[boolean(@disabled)=false()]`	XPath has no true/false literals — they are functions. Example = same as not(@disabled).
`lang(code)`	*boolean*	`//p[lang('en')]`	Checks inherited xml:lang; rarely useful in HTML UI tests.

4 Number functions

Function	Returns	Example	Notes
`number(obj?)`	*number*	`//td[number(.) > 100]`	Non-numeric text becomes NaN — and every comparison with NaN is false.
`sum(node-set)`	*number*	`sum(//td[@class='qty'])`	Niche in UI locators; handy when asserting over XML responses.
`floor(n)`	*number*	`//div[floor(@data-rating) = 4]`	Round down. floor(4.7) = 4.
`ceiling(n)`	*number*	`//div[ceiling(@data-progress) = 100]`	Round up. ceiling(99.2) = 100.
`round(n)`	*number*	`//div[round(@data-rating) >= 4]`	Nearest integer; .5 rounds up.

That's the entire XPath 1.0 library — 27 core functions — and all 27 work in every browser and every Playwright version.

5 text() vs "." vs string(.)

text() is technically a node test (like node() or comment()), not a function — but every tester meets it in predicates, so it belongs here.

- `text()` selects **direct child text nodes only** — //div[text()='Total'] fails if "Total" sits inside a nested .
- `.` (or string(.)) sees **all descendant text** — //div[contains(., 'Total')] is far more forgiving.
- Leading/trailing whitespace breaks `text()='...'` exact matches — wrap with normalize-space() instead.

Rule of thumb: normalize-space() for exact matches · contains(., ...) for partial matches · raw text() only when you deliberately want direct-child text.

6 Not in XPath 1.0 — these FAIL in Playwright

XPath 2.0+ function	XPath 1.0 workaround
`ends-with(@id, '_btn')`	`substring(@id, string-length(@id) - string-length('_btn') + 1) = '_btn'`
`lower-case() / upper-case()`	`the translate() mapping — see pattern #1 below`
`matches(., 'regex')`	`no pure-XPath option — use page.getByText(/pattern/i) or locator.filter({ hasText: /.../ })`
`replace(), tokenize(), string-join()`	`extract with textContent() and handle it in JS/TS`

These 2.0 functions do work in XSLT 2.0 processors and some API-testing tools — which is exactly why snippets copy-pasted from the internet break in browsers.

7 10 battle-tested patterns (copy-paste ready)

1. Case-insensitive match (the translate trick)

```
//a[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'logout')]
```

2. Exact visible text, whitespace-proof

```
//button[normalize-space()='Add to cart']
```

3. Dynamic IDs: anchor only the stable parts

```
//input[starts-with(@id, 'input-') and contains(@id, '-email')]
```

4. The nth-element gotcha

```
(//div[@class='card'])[2]    -> 2nd card on the page
//div[@class='card'][2]      -> 2nd card within EACH parent (usually not what you want)
```

5. From a label to its field

```
//label[normalize-space()='Email']/following-sibling::input[1]
```

6. Table cell lookup by row text

```
//tr[td[normalize-space()='Pramod']]/td[3]
```

7. Click an SVG icon

```
//*[name()='svg' and contains(@class, 'close')]
```

8. Hop to the parent

```
//input[@id='email']/..      (in code: locator.locator('xpath=..') or locator.locator('..'))
```

9. Only non-empty fields

```
//input[string-length(normalize-space(@value)) > 0]
```

10. Row ranges (skip the header row)

```
//tr[position() > 1 and position() < 7]      -> rows 2-6
```

XPath 1.0 function reference for UI automation · valid in Playwright, Selenium & Appium · prepared for Pramod Dutta — The Testing Academy